Building Agent Hub with AI

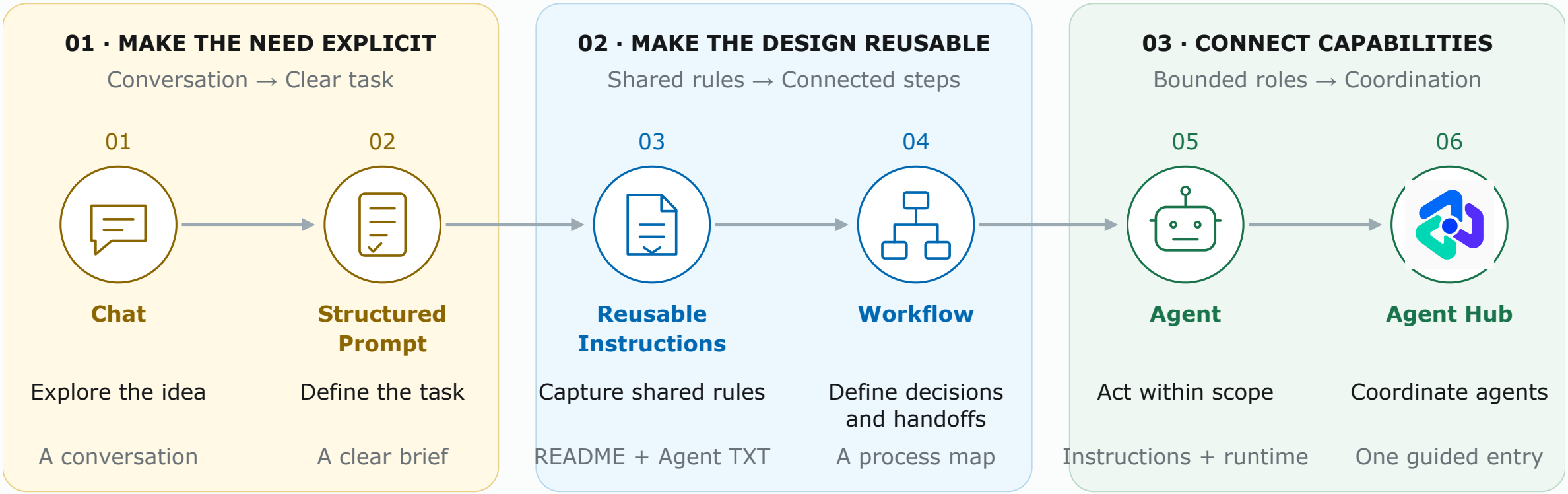
From idea to files: communicating with AI, defining standards, and iterating

SPEAKER: **XINGYU YOU** DATE: **2026-10-20**

01 · THE BIG PICTURE

From Chat to Agent Hub: the whole journey at a glance

Follow one project from an initial conversation to reusable rules and coordinated agent roles.



> A prompt becomes part of an agent design. Runtime, tools, state handling, and verification make execution possible.

Conceptual roadmap, not a strict creation timeline. Project definitions do not by themselves prove deployed capability.

02 · ROLE DESIGN

Start with the work, then define the agent roles

Identify the work within each user need, then assign responsibility.

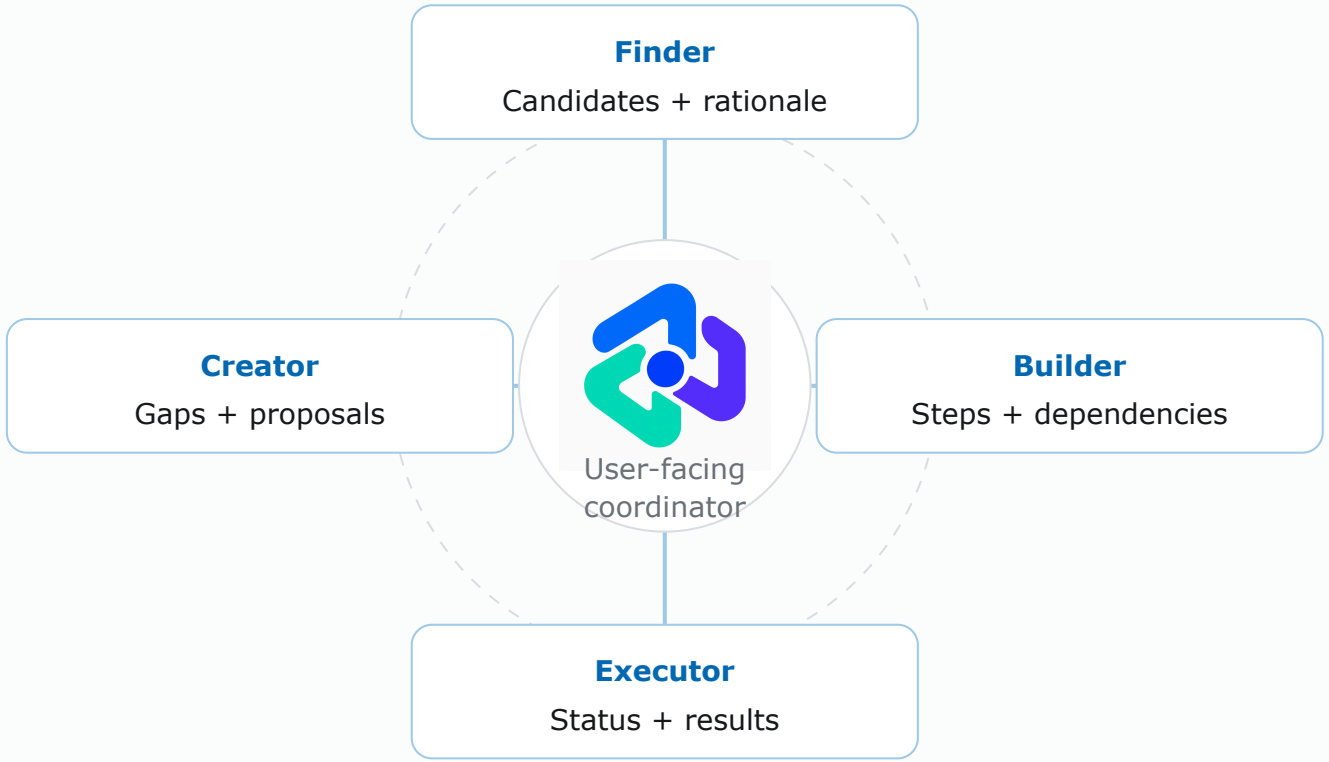
"I want one entry point to guide me from my goal to a result."	→	**Main Agent · Hub** Understand, route, confirm, summarize
"Who can do it? How do we combine them? Who runs it? What is missing?"	→	**Functional Agents** Finder / Builder / Executor / Creator
"Different tasks need different business capabilities."	→	**Business Agents** Task-specific inputs / Rules / Deliverables

Example prompt to AI

Derive roles from user needs. Specify each role's inputs, outputs, boundaries, and handoffs. Check for overlapping or unassigned responsibilities.

02 · ROLE DESIGN

Agent Hub at the center, with clear functional boundaries



01 · Hub owns the conversation

Understand the goal, route work, confirm scope, and summarize results.

02 · Functions return deliverables

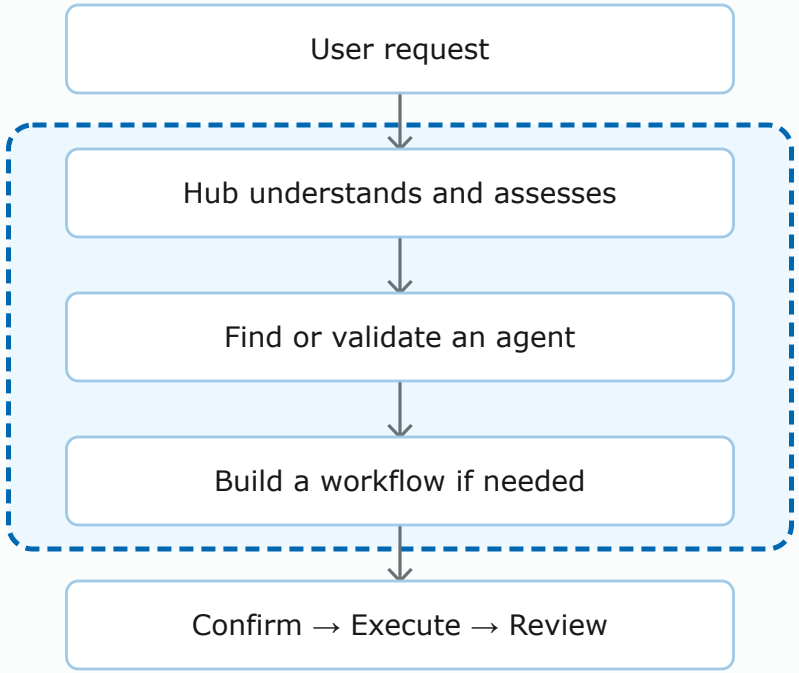
Recommendations, blueprints, gap proposals, and execution status.

03 · Business agents do the task

Executor dispatches authorized work and collects outputs.

This is a relationship map, not a fixed call sequence.

Workflow: map the main path, then add decisions



01 · Map the main path

Start with the user request and the expected result.

↓

02 · Add decision points

What is missing? Which agent fits? Is a multi-step plan needed?

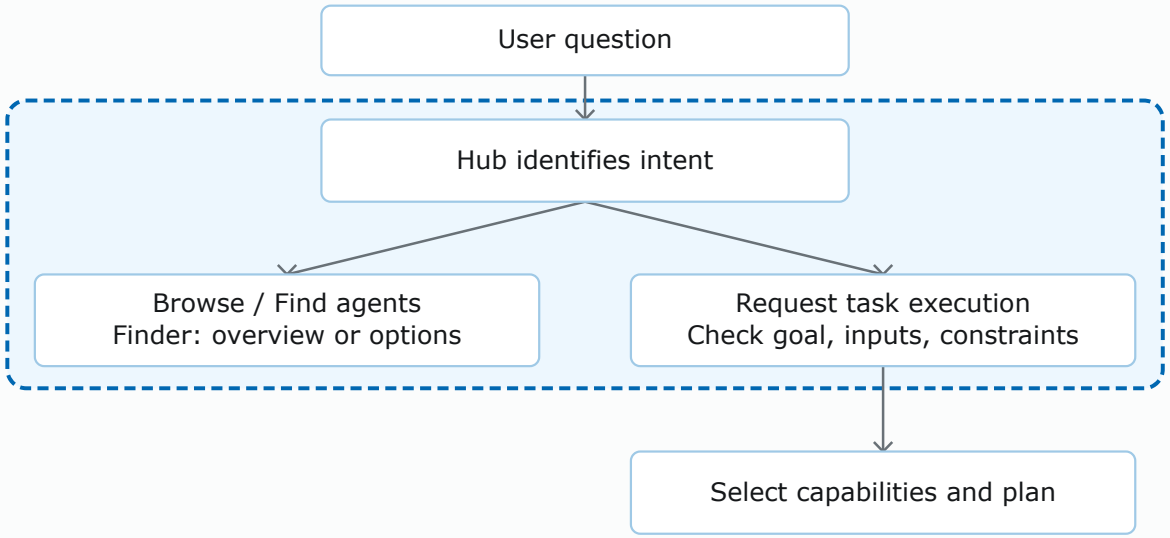
↓

03 · Capture the rules

Define branches, handoffs, and stopping conditions with AI.

Shared design → README / Task Routing Logic
Entry-point instructions → Agent-Hub.txt

Fragment 1: identify what the user wants



What does this part cover?

Possible user questions:

"What agents are available?"
"Compare these two documents."

Decisions to define:

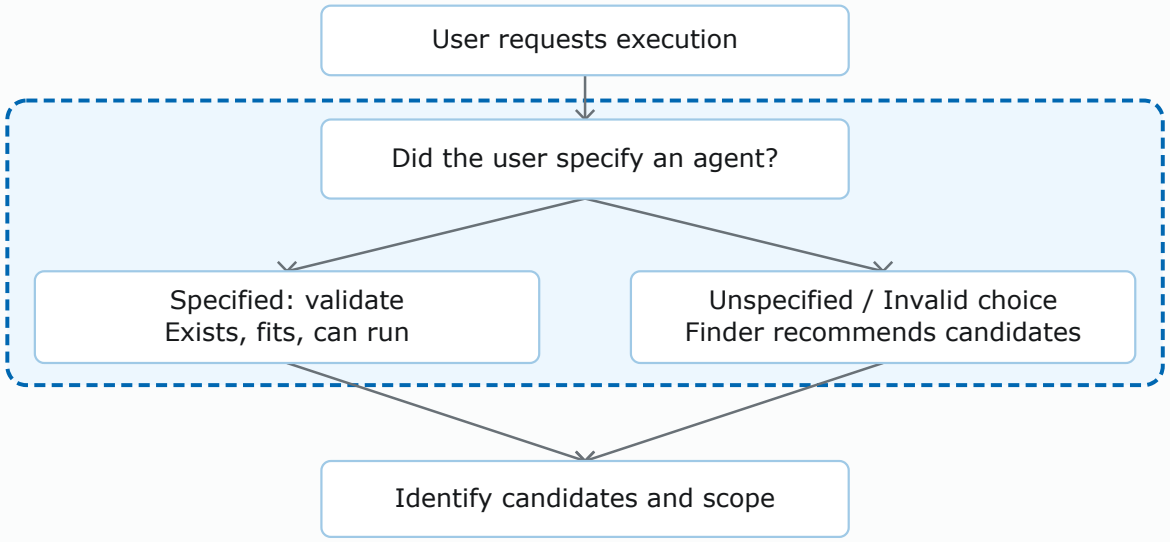
Is the user exploring capabilities or requesting execution? Are the goal and required inputs clear?

Rules to give AI:

Return an overview for browsing. Check inputs for execution. Clarify a missing goal rather than inventing one.

README.md → Clarification Rules / Task Routing Logic
Agent-Hub.txt → Core responsibilities

Fragment 2: find the right agent



What does this part cover?

Two entry paths:

Validate a user-specified agent. Otherwise, ask Finder for recommendations.

Conditions to consider:

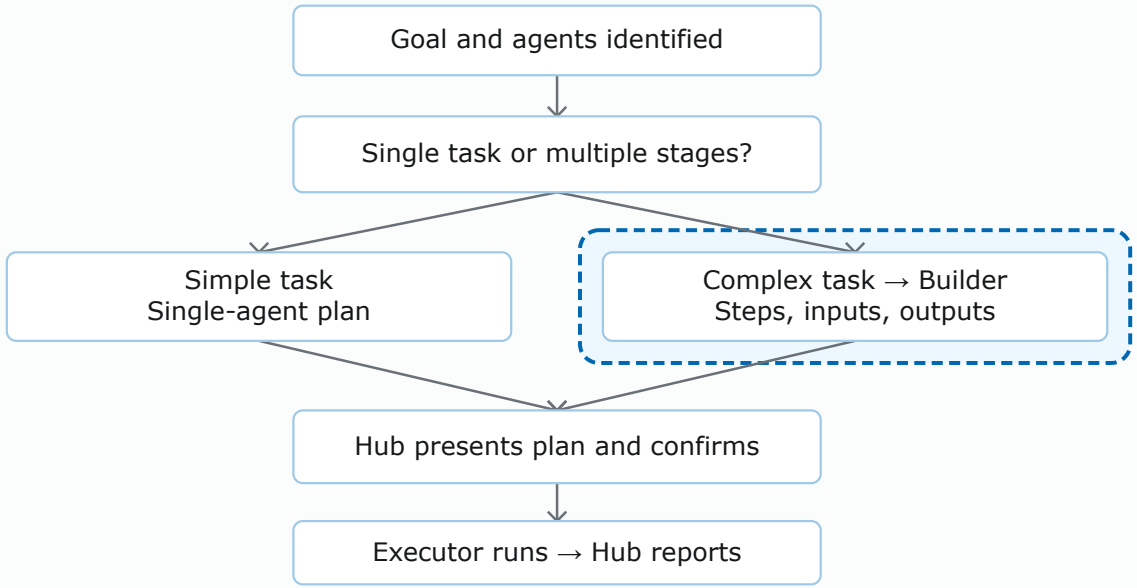
Check existence, capability fit, input compatibility, runtime availability, and permissions.

Rules to give AI:

Recommend alternatives for an invalid choice. If no match fits, report the gap and consider composition or creation.

README.md → Subagent Selection Rules
Agent-Hub.txt / functional-agents/Agent-Finder.txt

Fragment 3: break complex work into clear handoffs



Focus: Workflow Builder

Design question:

"If a request needs multiple agents, how should Agent Hub coordinate them?"

The blueprint must define:

Who owns each step, its inputs and outputs, and which prior steps it depends on.

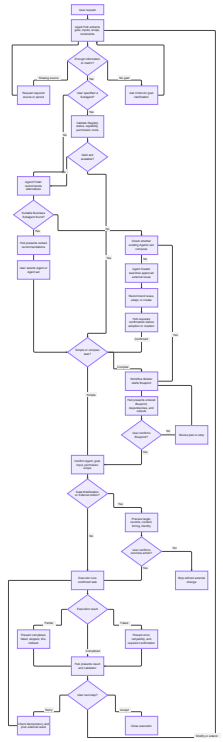
Rules to give AI:

Give Builder responsibility for steps and dependencies. Give Executor responsibility for authorized scheduling. Define their shared blueprint contract.

README.md → Complex Workflow Execution
functional-agents/Workflow-Builder.txt → steps
input / output / dependencies / permission_scope

03 · COMPLETE WORKFLOW

The complete Agent Hub flow: keep every branch connected



Click to explore the full diagram

01 · Understand and match

Missing information, specified agents, recommendations, and capability gaps

↓

02 · Plan and confirm

Simple or complex tasks, blueprint revision, and scope confirmation

↓

03 · Execute within scope

Concrete action preview, authorization, and execution

↓

04 · Review and continue

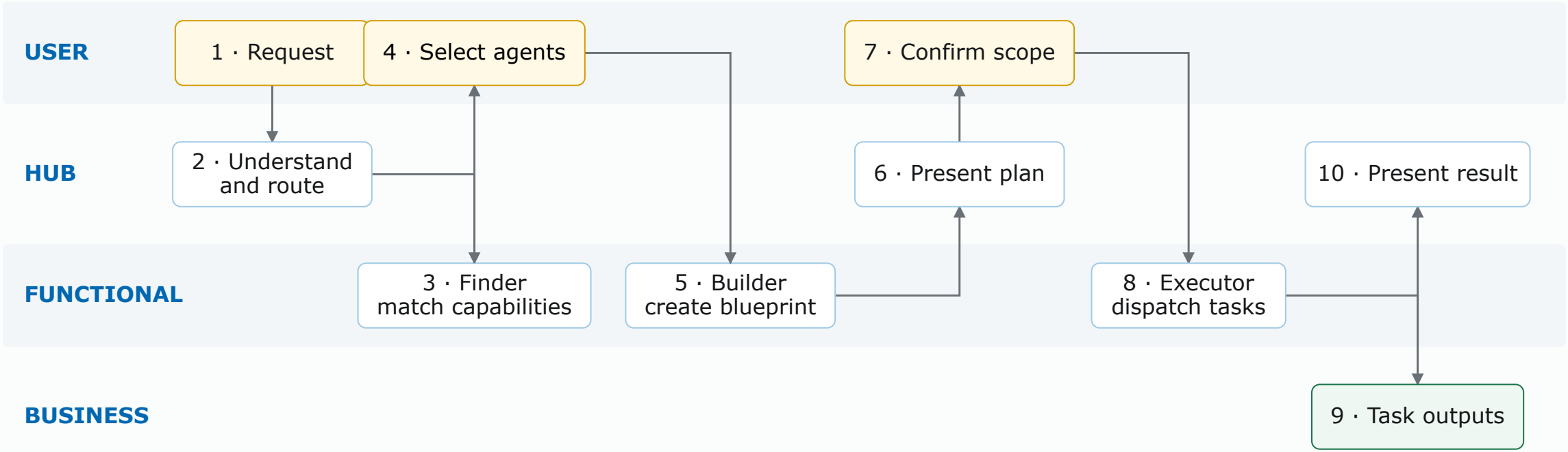
Complete, partial, or failed results, retry checks, and changed requests

When revising with AI: locate the branch, its return path, and unchanged rules.

Actual project diagram referenced by README → End-to-End Agent Hub Flow. Overview is for orientation. Open the zoomable view for readable labels. Copy canvas expanded to include source nodes beyond its original bounds.

03 · WORKFLOW

Who does what? Follow one confirmed workflow across lanes



Illustrative complex-task path: select recommended agents, then confirm the blueprint before execution. Missing inputs or new actions can pause the workflow.

Based on README routing and functional-agent boundaries. Arrows show handoffs, not measured timing. Agents not needed for this path are omitted.

04 · IMPLEMENT WITH AI

Inside our README: architecture and definition examples

Architecture · Who belongs where?

Core Architecture / grouped for presentation

```
User
└── Agent Hub
    ├── functional-agents/
    │   ├── Agent Finder
    │   ├── Workflow Builder
    │   ├── Agent Executor
    │   └── Agent Creator
    ├── Agent Registry
    └── business-subagents/
```

One Hub + four functional agents. Registry and business agents remain separate.

Definition template · What should AI write?

README.md / Definition Standard

```
# English Name

## Description

Describe the agent's core role and when it should be used.

## Instruction

Define responsibilities, boundaries, trigger conditions, workflow
```

A repeatable structure for generating and reviewing definitions.

> Example request: “Read these sections first. Keep the existing roles and format, and identify any changes to the agreed design.”

Left: project architecture regrouped by responsibility. Right: README template excerpt. These are document previews, not platform screenshots.

Why this structure? Give each type of information a home

A design rationale derived from the current project, rather than a claim about its original creation order.

AI needs shared context Preserve goals and shared rules.	→	**README.md** Project-wide agreements
Roles change independently Update one role independently.	→	**Main / Functional / Business** Separate TXT files by responsibility
Finder needs a searchable directory Find and compare existing capabilities.	→	**Agent-Registry.txt** Discovery metadata aligned with TXT
Progress and evidence need tracking Track tasks and verification separately.	→	**TODO.md / submission-materials/** Next actions and test evidence

172 definitions · 8 categories



Registry counts, cross-checked with TXT definitions. Labels shortened. Not deployed-agent counts.

Example prompt: “Propose the smallest file structure for these roles. Explain what each file owns and which files must stay synchronized.”

04 · IMPLEMENT WITH AI

From shared standards to one agent definition

Root README

Shared structure, TXT format, length limits

Business folder README

Names, required inputs, metadata consistency

Return + presentation rules

Structured results upstream, clear output through Hub

→

Example-Agent.txt

Name

Identify the capability.

Description

Explain when to choose it.

Instruction

- **WHEN** Triggers and inputs
- **DO** Steps and boundaries
- **RETURN** Results, status, and missing fields

Description ≤ 800 characters. Entire TXT < 8,000 characters. Missing required input → clarify before dependent execution.

05 · ITERATE & VALIDATE

One routing requirement: define → implement → challenge

> **Request to AI · Example**
>
> If the user specifies an unsuitable agent, explain the issue and suggest an alternative.

Define the branch

- Valid → Confirm scope
- Invalid → Finder alternatives

→

Record the rule

README.md

> Subagent Selection Rules

→

Implement it

Agent-Hub.txt

> Routing and confirmation

Challenge 1 · "Use this unavailable agent."

Explain the issue and recommend an alternative.

Challenge 2 · "Use it. Files will follow."

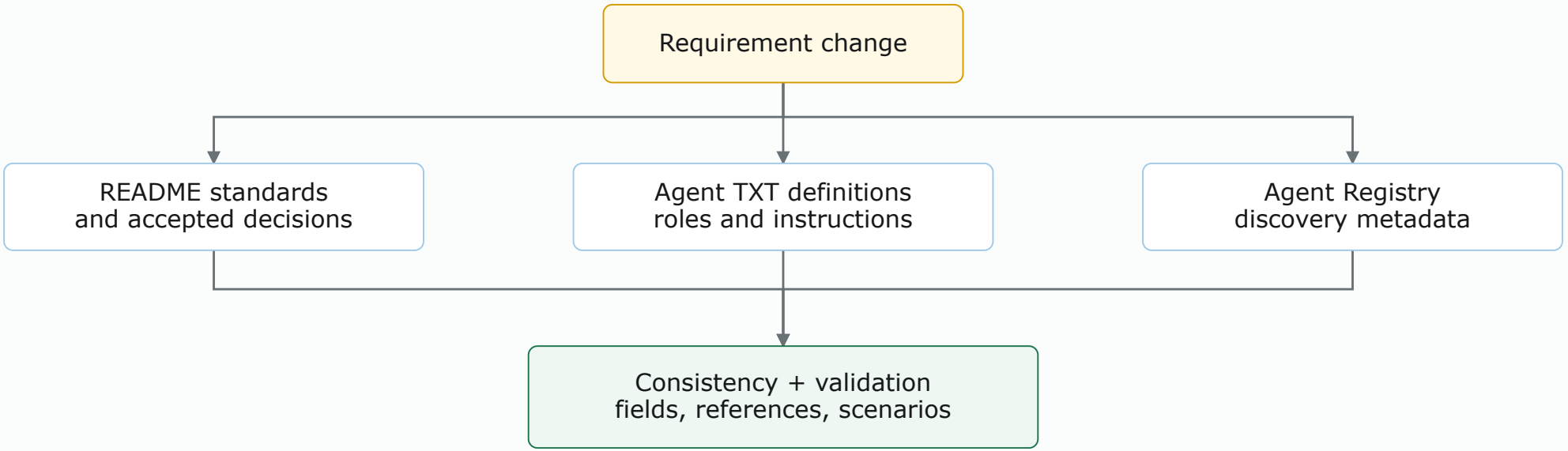
Validate the agent, then wait for required inputs.

Based on current project rules. Reconstructed prompt and walkthrough examples, not original dialogue or verified runtime results.

A file dependency map: one request can affect several files

Example feedback

Use one English paragraph, within 800 characters. Keep capability, triggers, and essential limits.



A change can affect several files. Update only the relevant dependencies.

Definitions and Registry metadata must agree. Relevant test scenarios and TODO items should reflect the changed rule.

05 · ITERATE & VALIDATE

Active is a definition status, not runtime proof

Documented correction · TODO, 2026-10-15

Active means the TXT definition is complete and discoverable. It does not mean deployed or runtime-validated.

01 · Definition

- Roles and boundaries
- TXT and Registry agree

Active applies here

→

02 · Runtime

- Tools and permissions
- Actual agent calls

Requires run evidence

→

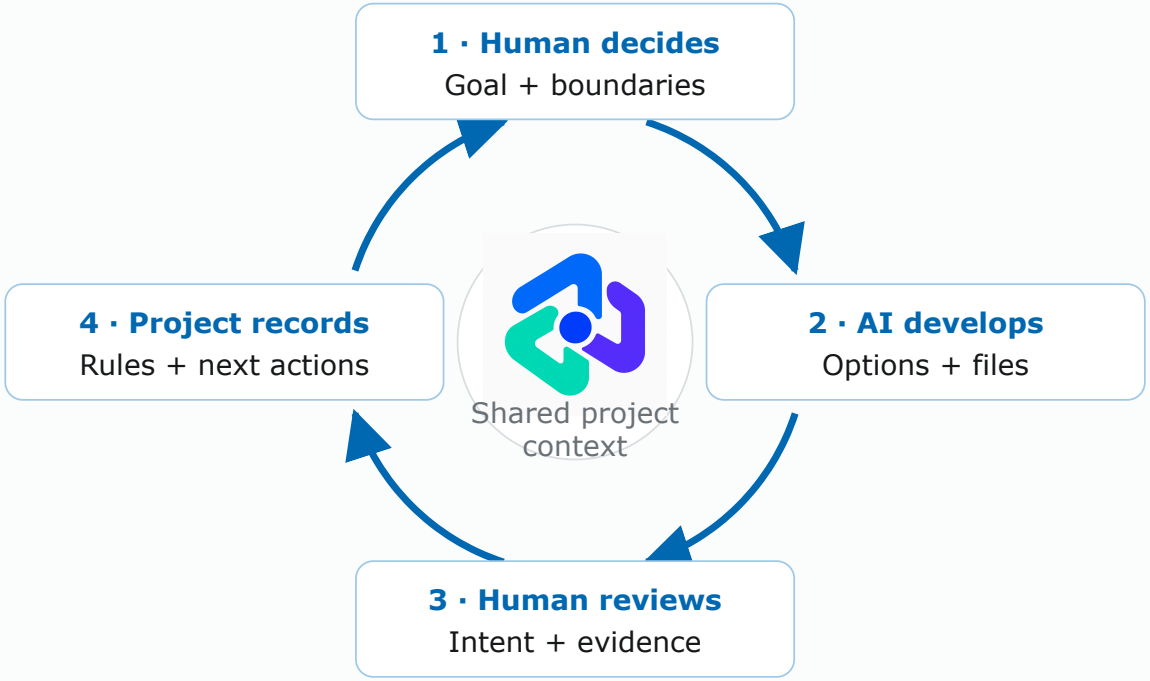
03 · Results

- Source support
- Acceptance criteria

Requires output review

The correction is recorded in TODO.md. These are separate validation gates, not a claim that all checks have passed.

A visible loop: decide → develop → review → record



Each iteration leaves a trace

Decide what the project should achieve.

Ask AI to expand and implement the design.

Review the proposal and evidence.

Save accepted changes so the next request starts with shared context.

> A process loop, not a chart of time or effort.

06 · REAL FEEDBACK

Real feedback from this conversation changed the presentation

01 · Correct the scope

> “Isn’t Agent Hub itself the case that runs through the whole 1020 presentation?”

Resulting revision

Removed the separate SOP case section. Reconnected requirements, roles, workflow, files, and iteration to Agent Hub.

02 · Ask for the reasoning

> “The key is how we came up with this file structure.”

Resulting revision

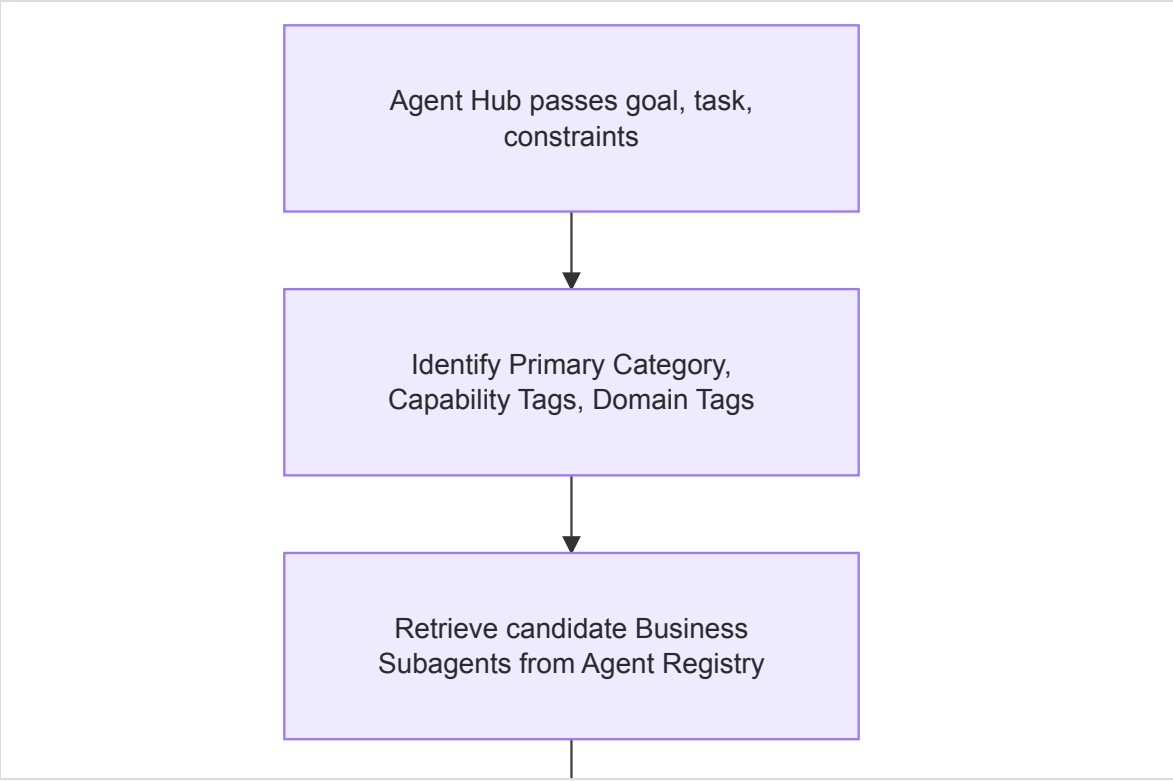
Replaced a directory listing with need → design rationale → file responsibility.

Useful feedback identifies the mismatch, restates the intent, and asks for a concrete change.

Translated excerpts from this presentation-preparation conversation. These are real editorial corrections, not evidence of the original Agent Hub development history or runtime behavior.

06 · KEEPING CONTEXT

When AI revisits the project: use the diagram as a checkpoint



Original Finder diagram, cropped to its first three steps. Open the complete diagram.

1. Read the current README, relevant diagram, and agent instructions.
2. Locate the affected node or branch before changing the design.
3. Update the rule, diagram, and affected TXT or Registry together.
4. Walk through the changed branch. Report conflicts and missing decisions.

> Example: “Keep the Finder sequence: identify tags → retrieve candidates → evaluate fit. Explain any proposed deviation before applying it.”

These artifacts help expose inconsistencies. They do not guarantee that AI reads them automatically or that runtime behavior is correct.

06 · PRACTICAL TAKEAWAYS

A reusable communication template: design the workflow and README

01 · Goal

What should the project achieve?

02 · Context

Which ideas, files, and decisions should AI use?

03 · Scope

Which workflow and README sections should it produce?

04 · Constraints

Which boundaries must remain? What is still undecided?

05 · Acceptance

What outputs and checks will make the result reviewable?

EXAMPLE · ONE COMPLETE REQUEST TO AI

Goal. Help me design Agent Hub, a single entry point for finding and coordinating business agents.

Context. Use my requirements and available project files. First summarize accepted decisions and open questions.

Scope. Map likely user requests into a workflow. Define Hub, functional, and business-agent responsibilities. Propose a minimal file structure and draft the README: vision, architecture, routing, definition standards, decisions, and open questions.

Constraints. Keep recommendation, planning, and execution separate. Require confirmation before execution. Flag missing decisions. Do not invent tools, deployment, or test results.

Acceptance. Return the workflow diagram, file responsibilities, and README draft. Walk through representative user questions and list conflicts, assumptions, and next steps.

For an existing project: read the current README and diagram first, explain the proposed changes, then synchronize the affected files.

Reusable example derived from Agent Hub's design. Adapt the five elements to the task. This is not a verbatim historical prompt.

TAKEAWAY

From a conversation to a shared, reviewable design

01

Design the logic

Define roles, decisions, and handoffs before expanding the system.

02

Keep a shared reference

Capture accepted rules in README, diagrams, and agent instructions.

03

Review every change

Check affected files and separate written definitions from verified execution.

Capture the idea as context.

Capture requirements as standards.

Use files to guide AI through implementation and review.